

Botnet Detection Using Long Short-Term Memory Networks and Fully Connected Neural Networks with Advanced Preprocessing on the CTU-13 Dataset

1st Ahmad Zaki

Department of Informatics Engineering
Brawijaya University
Malang, Indonesia
ahmadzaki12@student.ub.ac.id

2nd Arion Syemael Siahaan

Department of Informatics Engineering
Brawijaya University
Malang, Indonesia
siahaanarion@student.ub.ac.id

3rd Muhammad Razi Al Kindi Nadra

Department of Informatics Engineering
Brawijaya University
Malang, Indonesia
alkindirazi@student.ub.ac.id

Abstract— Botnets pose a significant cybersecurity threat, conducting malicious activities such as distributed denial-of-service (DDoS) attacks and data theft. This study explores deep learning techniques, specifically Long Short-Term Memory (LSTM) networks and Fully Connected Neural Networks (FCNN), for detecting botnet traffic in the CTU-13 dataset. The dataset's challenges, such as class imbalance, are addressed using advanced methods including SMOTETomek, Focal Loss, and regularization. The models are evaluated using precision, recall, F1-score, and PR AUC metrics. Among the proposed models, the Stacked LSTM achieved the highest PR AUC of 0.8326 and an F1-score of 64%, demonstrating superior performance in handling imbalanced data and identifying botnet traffic. While precision remains an area for improvement, the findings validate the efficacy of deep learning in enhancing cybersecurity. This research contributes to the field by providing scalable, data-driven approaches for proactive botnet detection, with implications for improving real-world network security frameworks.

Keywords— Botnet Detection, Deep Learning, LSTM, FCNN, CTU-13 Dataset, Cybersecurity.

I. INTRODUCTION

Cybersecurity is a critical aspect of modern digital infrastructure, especially with the increasing volume and complexity of cyberattacks. Threats such as botnets, malware, and Distributed Denial of Service (DDoS) attacks exploit vulnerabilities in network systems, posing significant risks to organizations and individuals. The growing reliance on digital systems for business operations, communication, and personal use has made securing these systems a priority. However, cybersecurity efforts face challenges due to the evolving nature of threats, which are becoming increasingly automated and coordinated, as exemplified by botnet-based attacks. Botnets, consisting of interconnected compromised devices, are capable of launching large-scale attacks that can evade traditional detection mechanisms. This underscores the importance of exploring advanced, data-driven solutions for proactive threat detection and mitigation. Traditional detection methods often fall short in identifying sophisticated attacks due to their inability to capture complex patterns in network traffic. Deep learning has emerged as a promising approach for addressing these challenges, leveraging its capacity to learn and generalize from intricate data patterns.

This study focuses on leveraging deep learning techniques, particularly Bidirectional Long Short-Term Memory (Bi-LSTM) and Fully Connected Neural Networks (FCNN), to improve the detection of cyber threats. The CTU-13 dataset, which comprises real network traffic, serves as the foundation for this research. However, the dataset presents challenges, such as class imbalance between normal traffic and attack data, which necessitate the use of advanced methods like Synthetic Minority Oversampling Technique (SMOTE) and Focal Loss to enhance model performance.

The objectives of this research are threefold: first, to develop robust deep learning models capable of detecting cyber threats; second, to explore the effectiveness of supporting techniques such as SMOTE, Focal Loss, and K-fold cross-validation in mitigating data and training challenges; and third, to evaluate the performance of these models using metrics such as accuracy, precision, recall, and Area Under Curve (AUC).

While this research provides insights into the application of deep learning for cybersecurity, it operates within certain constraints. The use of the CTU-13 dataset restricts the findings to the specific network environment it represents, such as a university network. Additionally, this study focuses solely on detection models without extending to real-time implementation or attack mitigation strategies. Finally, computational resources impose limitations on the scale and complexity of the models developed.

II. RELATED WORK

Botnet detection has been extensively explored using a variety of methods. Traditional approaches, such as signature-based and anomaly detection systems, have shown certain limitations. Signature-based methods are highly effective for known threats but fail against evolving or novel botnets, while anomaly detection often suffers from high false-positive rates, particularly in complex and dynamic network environments [1][2]. These challenges have driven researchers toward adaptive solutions like machine learning (ML) and deep learning (DL).

Machine learning algorithms, including decision trees, random forests, and support vector machines (SVM), have demonstrated success in automating feature selection and classification tasks. However, these methods typically require manual feature engineering and struggle to capture the temporal patterns inherent in network traffic [3][4].

Deep learning techniques, particularly Long Short-Term Memory (LSTM) networks, have emerged as a promising alternative. LSTMs excel at capturing temporal dependencies in sequential data, making them highly suitable for analyzing time-series data such as network flows. Research has consistently demonstrated the superior performance of LSTMs in detecting botnet activities compared to traditional ML methods [5][6].

Hybrid deep learning models, such as CNN-LSTM architectures, have also shown promise by integrating spatial and temporal feature extraction, leading to more robust detection capabilities [7]. For instance, CNN layers capture spatial features in network traffic, while LSTM layers analyze temporal dependencies, resulting in improved botnet detection performance. Similarly, Bidirectional LSTMs (Bi-LSTM) have been utilized to further enhance temporal feature analysis by processing data in both forward and backward directions, offering a more comprehensive understanding of sequential patterns [6].

Class imbalance remains a persistent challenge in botnet detection, as normal traffic typically outweighs malicious traffic in datasets like CTU-13. To address this, researchers have employed various oversampling techniques, such as SMOTE, SMOTETomek, and SMOTEENN, to balance the class distribution [3][8]. Advanced loss functions like Focal Loss have also been implemented to mitigate bias toward majority classes, improving the detection of minority class instances. Recent studies have highlighted the importance of combining oversampling with cost-sensitive loss functions to create more effective detection models, particularly for imbalanced datasets [9].

Another emerging trend involves the integration of Automated Machine Learning (AutoML) frameworks to address challenges such as class imbalance and hyperparameter optimization. These frameworks simplify the process of model selection and training, enabling the development of high-performance models with minimal manual intervention. For example, AutoML has been used to develop cost-sensitive loss functions and optimize sampling techniques, achieving significant improvements in computational efficiency and detection accuracy [9].

The effectiveness of sampling techniques has been explored beyond botnet detection as well. For instance, a systematic review by Saleem et al. demonstrated the enhancement of CNN performance in automated defect detection tasks using sampling techniques for imbalanced datasets. This study emphasized the role of advanced oversampling methods in improving model robustness and generalizability, insights that are directly applicable to botnet detection scenarios [9].

Despite these advancements, certain limitations persist. Many existing methods require large and diverse datasets to ensure robustness and generalizability. High computational costs often accompany deep learning-based models, particularly when addressing the complexity of temporal dependencies and class imbalances. Additionally, achieving consistent performance across unseen botnet types remains a challenge. Recent studies have emphasized the need to integrate advanced preprocessing techniques with DL architectures, such as Bidirectional LSTM and Fully Connected Neural Networks (FCNN), to overcome these limitations. These approaches aim to create scalable and

adaptable solutions for real-world cybersecurity applications [6][9].

III. PROPOSED SOLUTION

A. Solution Description

This research focuses on detecting botnet activity within network traffic by leveraging deep learning models, particularly Bidirectional Long Short-Term Memory (Bi-LSTM) networks and Fully Connected Neural Networks (FCNN). The combination of these models with advanced techniques such as SMOTETomek, Focal Loss, and feature engineering aims to overcome challenges like class imbalance, noisy data, and temporal dependencies inherent in network traffic data.

The solution employs comprehensive data preprocessing, sequence preparation for LSTM models, and advanced optimization strategies. Evaluation metrics such as precision, recall, F1-score, and ROC-AUC are used to assess the effectiveness of the models.

B. Dataset

The dataset used in this research is the CTU-13 dataset, a well-known benchmark in botnet detection studies. This dataset contains real-world network traffic data labeled as either normal or botnet traffic, providing an ideal basis for supervised learning approaches. Below are the key features and components of the CTU-13 dataset:

1. StartTime: The timestamp indicating when the network flow began.
2. Duration (Dur): The total duration of the network flow in seconds.
3. Protocol (Proto): The protocol used for communication, such as TCP or UDP.
4. Source Address (SrcAddr): The IP address of the traffic source.
5. Source Port (Sport): The port number used by the traffic source.
6. Direction (Dir): The flow direction, indicating whether the traffic is inbound or outbound.
7. Destination Address (DstAddr): The IP address of the traffic destination.
8. Destination Port (Dport): The port number used by the traffic destination.
9. State: The state of the flow, providing information about its status (e.g., SR_SA, SRA_SA).
10. Service Type (sTos, dTos): Indicators of the type of service used in the traffic.
11. Total Packets (TotPkts): The total number of packets transmitted during the flow.
12. Total Bytes (TotBytes): The total amount of data (in bytes) transmitted during the flow.
13. Source Bytes (SrcBytes): The amount of data (in bytes) sent by the source.
14. Label: The ground truth classification for the flow, such as "Background-TCP-Established" or other specific botnet-related labels.

This bar chart displays the distribution of the classes in the dataset, with a significant imbalance between normal (class 0) and botnet (class 1) traffic. Normal traffic dominates, emphasizing the need for techniques like oversampling to address the imbalance during model training.

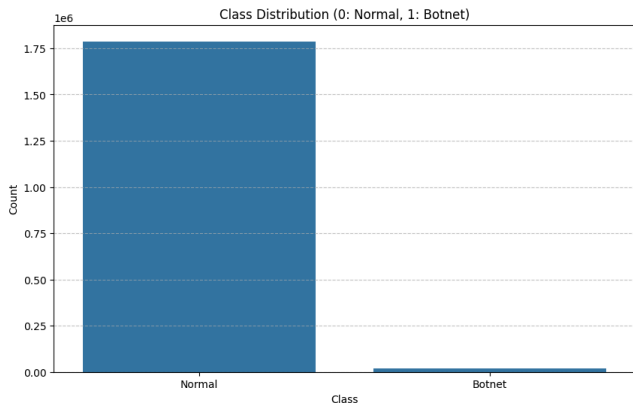


Fig. 1. Class Distribution

The histogram represents the distribution of the Dur (duration) feature. Most network flows, especially those associated with botnets, are concentrated within very short durations, reinforcing the characteristic behavior of botnet traffic as rapid and repetitive.

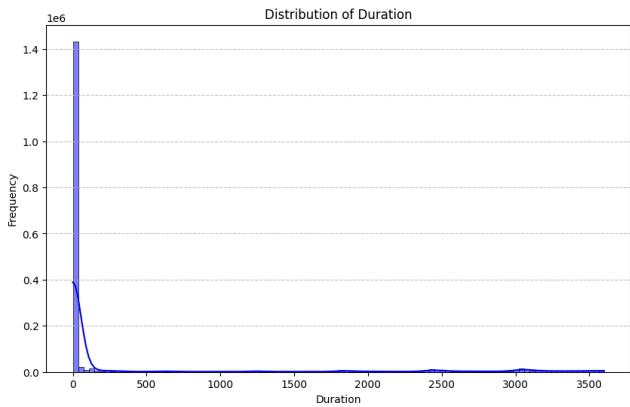


Fig. 2. Distribution of Duration

This pairplot shows pairwise relationships between features like Dur, TotPkts, TotBytes, and SrcBytes. Distinct clusters can be observed for botnet and normal traffic, particularly in features like TotBytes and SrcBytes, which show clear separations, emphasizing their importance in classification tasks.

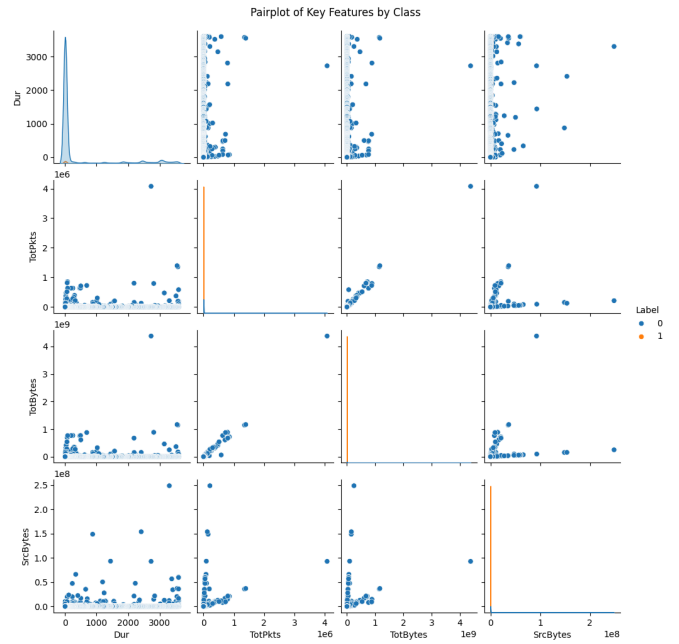


Fig. 3. Pairplot of Key Features

The heatmap reveals strong correlations between 'TotPkts' and 'TotBytes' (0.99), indicating that the total number of packets is closely tied to the total bytes transmitted in a flow. A moderate positive correlation (0.36) is observed between 'SrcBytes' and both 'TotPkts' and 'TotBytes', showing that source bytes contribute to overall traffic but to a lesser extent. In contrast, 'Dur' shows minimal correlation with other features, highlighting its independence and potential to capture unique aspects of network traffic behavior. These relationships help guide feature selection and engineering for model optimization.

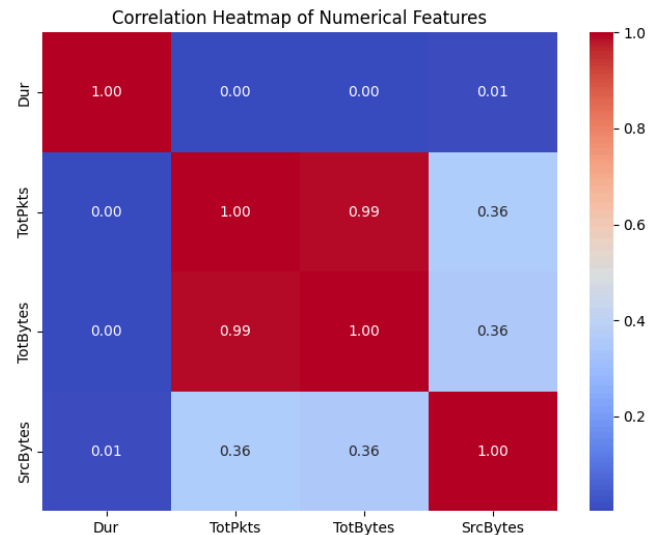


Fig. 4. Correlation Heatmap of Numerical Features

This figure illustrates the distribution of duration (Dur) for network traffic based on two classes: normal (0) and botnet (1). The plot indicates that botnet traffic tends to have shorter durations compared to normal traffic, with a few outliers showing longer durations in normal traffic. This information is crucial for identifying typical communication patterns of botnets, which usually involve short-lived interactions.

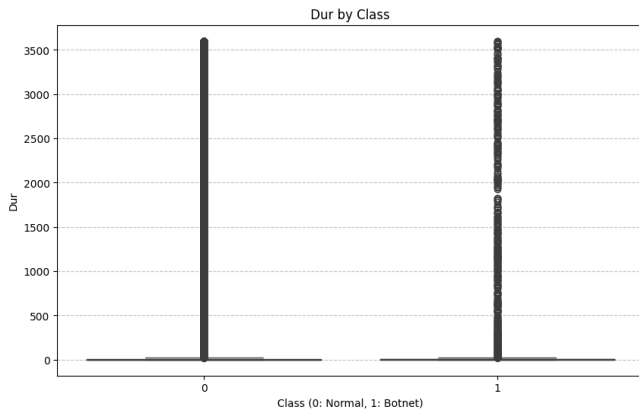


Fig. 5. Boxplot of Duration by Class

This figure visualizes the distribution of the TotPkts feature across normal (class 0) and botnet (class 1) traffic. The boxplot shows that botnet traffic typically involves fewer packets, with minimal variability, whereas normal traffic has a wider range of packet counts with significant outliers. This highlights that botnet traffic tends to be lightweight and controlled, making it an essential feature for classification.

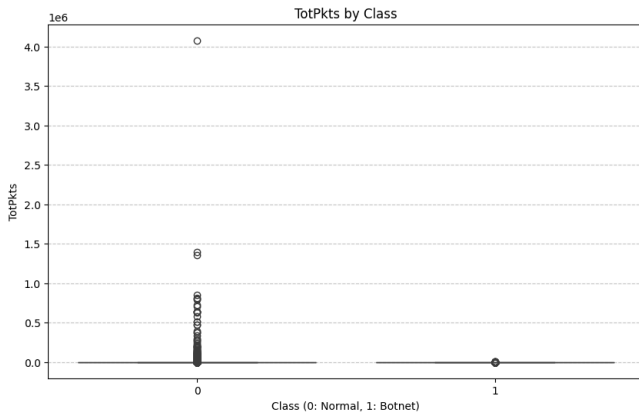


Fig. 6. Boxplot of Total Packets by Class

This figure depicts the distribution of total bytes (TotBytes) transferred during network traffic for the two classes: normal (0) and botnet (1). Botnet traffic consistently shows lower total bytes compared to normal traffic, with outliers only appearing in normal traffic. These results highlight the characteristic of botnets to avoid transferring large amounts of data to minimize detection risks.

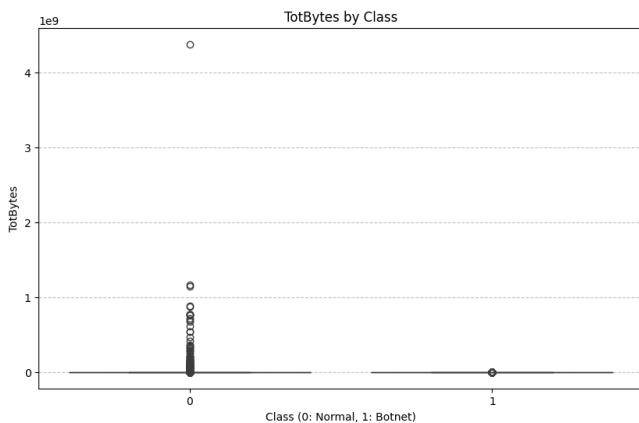


Fig. 7. Boxplot of Total Bytes by Class

This boxplot illustrates the distribution of SrcBytes for normal and botnet traffic. Botnet traffic is limited to a narrower range of source bytes, while normal traffic shows greater variability and significant outliers. This feature reflects the restricted nature of botnet communication.

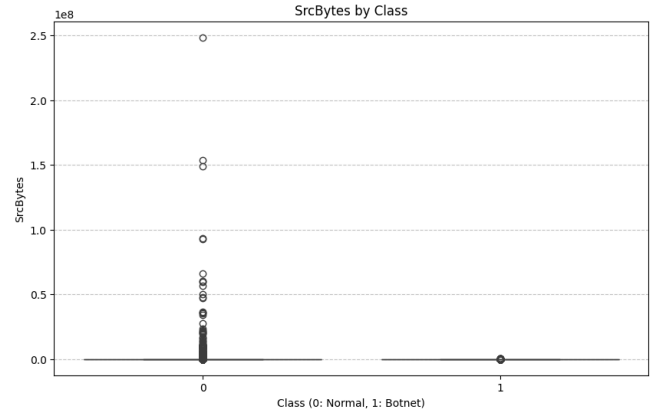


Fig. 8. Boxplot of Source Bytes by Class

These attributes make the CTU-13 dataset an essential resource for testing and improving botnet detection methodologies, particularly for models that rely on sequential patterns and flow-level analysis.

C. Methodology

1. Data Preprocessing

- Missing values in categorical columns (Sport and Dport) were replaced with 'unknown', while numerical columns (sTos and dTos) were filled with their respective medians.
- Invalid rows were removed, including those with Dur values less than 0, instances where TotBytes equaled 0 but TotPkts was greater than 0, and cases where SrcBytes exceeded TotBytes.
- Outliers in numerical columns (Dur, TotPkts, TotBytes, and SrcBytes) were managed using the Interquartile Range (IQR) method, with extreme values clipped to fall within $3 * IQR$.
- Applied RobustScaler to normalize numerical features for better convergence.
- Numerical columns were converted to absolute values to ensure all data entries were valid.
- Labels were binarized, assigning a value of 1 to instances containing 'Botnet' and 0 to all others.
- Duplicates were identified and removed to maintain dataset uniqueness.
- The cleaning process was evaluated by reporting the number of rows and missing values before and after cleaning, along with the class distribution percentages.

2. Feature Engineering

- Temporal features were derived from the StartTime column, including Hour, Day, Month, DayOfWeek, and IsWeekend, which indicates whether the instance occurred during a weekend.
- Network-related metrics were calculated, such as BytesPerPacket (total bytes divided by total packets), BytesPerSecond (total bytes divided by duration), and PacketsPerSecond (total packets divided by duration).
- Additional features included SrcBytesRatio (ratio of source bytes to total bytes), AvgDurPerPacket (average duration per packet), and

SrcToDstPktRatio (total packets divided by source bytes plus one).

- Encoding was applied to categorical columns (Proto, State, and Dir) using the mean of the Label column for each category, producing the Proto_encoded, State_encoded, and Dir_encoded features.

3. Model Architecture

- Bidirectional LSTM with Attention
 - Use a Bidirectional LSTM layer with 64 units to capture sequential dependencies in temporal data.
 - Apply an Attention layer to assign dynamic weights to the most relevant features.
 - Utilize a GlobalAveragePooling1D layer to reduce the dimensionality of the Attention output.
 - Add a dense layer with 32 units and ReLU activation for non-linear transformations, combined with L1L2 regularization.
 - Include a dropout layer with a rate of 0.4 to prevent overfitting.
 - Use a final dense layer with sigmoid activation for binary classification.

The Bidirectional LSTM with Attention architecture combines LSTM layers and an attention mechanism to effectively analyze temporal data. The Bidirectional LSTM captures sequential dependencies in both forward and backward directions, ensuring comprehensive temporal pattern learning. The Attention layer highlights the most relevant parts of the sequence by creating a context vector. This vector is then passed through GlobalAveragePooling1D to reduce dimensionality, followed by dense and dropout layers for feature extraction and overfitting prevention. Finally, the output layer uses a sigmoid function for binary classification, distinguishing between botnet and normal traffic.

Fig. 9 illustrates this process, showing the input sequence processed by the Bidirectional LSTM, enhanced by the Attention mechanism, and classified through subsequent layers for precise detection.

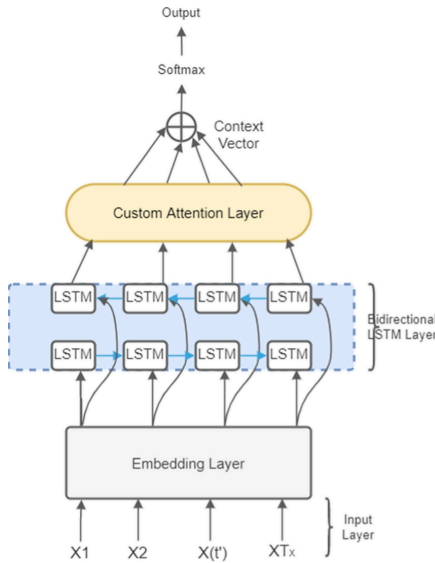


Fig. 9. Bidirectional LSTM with Attention Architecture

- Stacked LSTM with Regularization
 - Use two stacked LSTM layers; The first LSTM layer with 128 units and return_sequences=True to capture temporal dependencies; The second LSTM layer with 64 units to process higher-level sequential patterns.
 - Apply Batch Normalization after each LSTM layer for training stability.
 - Use dense layers with 32 units and ReLU activation, combined with L1L2 regularization.
 - Apply dropout with progressively decreasing rates (0.4 to 0.3) across layers to mitigate overfitting.
 - Use a final dense layer with sigmoid activation for binary classification.

Fig. 10 depicts a generalized architecture of a Stacked LSTM. It consists of multiple LSTM layers stacked hierarchically to capture both low-level and high-level temporal patterns. The data flows through the LSTM cells sequentially, where each layer processes the output from the previous layer. Dropout layers between the LSTM layers reduce overfitting by randomly deactivating neurons during training. Finally, dense layers aggregate the learned representations before producing the final binary output. This structure highlights the model's ability to process sequential data with increasing abstraction at each layer.

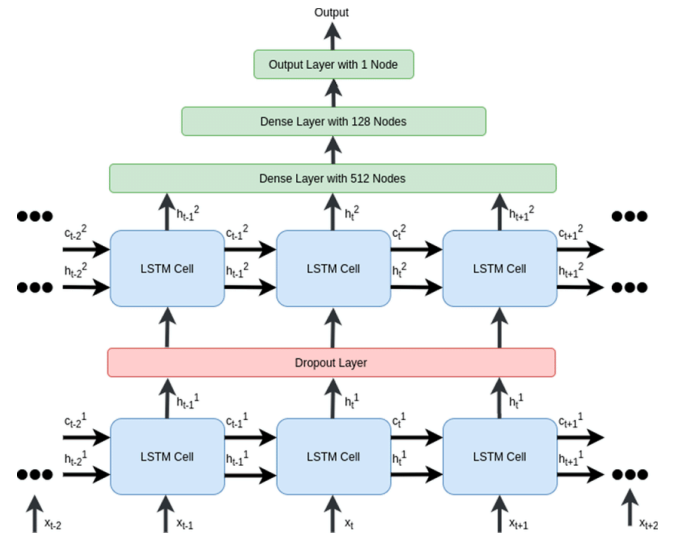


Fig. 10. Stacked LSTM Architecture

- Fully Connected Neural Network (FCNN)
 - Include an input layer accepting the preprocessed feature vectors.
 - Add a dense layer with 128 units, ReLU activation, and L1L2 regularization to extract high-dimensional feature interactions.
 - Use additional dense layers with 64 and 32 units, each followed by Batch Normalization and dropout layers (rates decreasing from 0.4 to 0.2).
 - Utilize a final dense layer with sigmoid activation for binary classification.

The architecture of a Fully Connected Neural Network (FCNN) consists of multiple layers where each neuron in a layer is fully connected to every neuron in the subsequent layer. This dense connectivity allows the network to learn complex relationships and high-dimensional feature interactions. The FCNN typically begins with an input layer for feature vectors, followed by hidden layers with nonlinear activations like ReLU to capture patterns, and ends with an output layer for prediction tasks such as binary classification. Regularization and dropout techniques are often applied to improve model generalization and prevent overfitting.

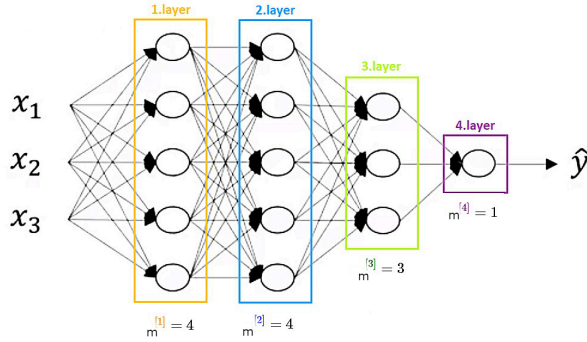


Fig. 11. FCNN Architecture

- Oversampling (SMOTE and SMOTETomek)
 - Apply Synthetic Minority Oversampling Technique (SMOTE) or SMOTETomek to balance the class distribution.
 - Reshape the oversampled data to match the input shape of the model (e.g., (batch_size, timesteps, features) for LSTM models).
- 4. Optimization
 - Use Focal Loss with the following parameters which mitigates class imbalance by reducing the influence of easy-to-classify examples.
 - Gamma: 2.0
 - Alpha: 0.25
 - Use the Adam optimizer with an Exponential Learning Rate Scheduler
 - Initial Learning Rate: 0.001
 - Decay Steps: 1000
 - Decay Rate: 0.9
 - Apply L1L2 regularization in dense and recurrent layers to improve generalization.
- 5. Training and Validation
 - Reshape the data into (batch_size, timesteps, features) format for LSTM-based models.
 - Apply oversampling techniques (SMOTE or SMOTETomek) to balance the training data.
 - Perform 5-fold Stratified Cross-Validation to ensure robustness and evaluate performance on diverse data splits.
 - Early Stopping stops training if validation loss does not improve for 5 consecutive epochs
 - ReduceLROnPlateau decreases the learning rate by a factor of 0.2 if validation loss stagnates for 3 epochs.
 - Set batch size dynamically (32 to 256) based on computational resources.

- Train for a maximum of 20 epochs, monitoring validation loss closely.

D. Differences from previous equations

The proposed solution sets itself apart from previous approaches by addressing critical challenges identified in related studies, such as class imbalance, temporal dependencies, and computational efficiency. Traditional methods, including signature-based and anomaly detection systems, often fail to detect novel or evolving botnets and suffer from high false-positive rates in complex network environments. Machine learning techniques, while effective in automating feature selection, struggle to capture temporal patterns in network traffic, limiting their adaptability to dynamic scenarios. Deep learning advancements, such as hybrid CNN-LSTM models, have shown promise by combining spatial and temporal feature extraction. However, these methods often introduce excessive computational complexity and higher resource requirements.

To overcome these limitations, the proposed solution utilizes three tailored architectures—Bidirectional LSTM (Bi-LSTM) with Attention, Stacked LSTM, and Fully Connected Neural Networks (FCNN)—optimized for temporal pattern recognition and feature interactions. By prioritizing temporal dependencies and feature representation rather than relying on complex hybrid structures, the solution delivers an efficient yet robust approach for botnet detection. Key advancements include advanced preprocessing and feature engineering techniques that enhance data richness through temporal feature extraction, network metric calculations, and categorical encoding. These improvements empower the models to better distinguish between normal and malicious traffic.

Additionally, the solution tackles class imbalance through refined oversampling techniques such as SMOTETomek and SMOTEENN, which eliminate noise introduced during traditional oversampling methods. Advanced loss functions, including Focal Loss, further address biases toward majority classes, improving the detection of minority instances. Regularization techniques, such as L1L2 regularization and dropout layers, combined with adaptive learning rate schedulers, prevent overfitting and enhance model generalization. Cross-validation ensures consistent performance across diverse validation folds, mitigating the variability seen in earlier methods.

The proposed architectures are also designed for scalability and adaptability, enabling their application across diverse network environments and datasets. Unlike prior methods requiring extensive manual intervention, this solution automates much of the optimization process, making it more suitable for evolving cybersecurity threats. While hybrid CNN-LSTM models introduce unnecessary complexity for botnet detection, the focus on Bi-LSTM with Attention and Stacked LSTM models streamlines the process by concentrating exclusively on temporal dependencies, thereby reducing computational overhead while maintaining high detection accuracy. The inclusion of FCNN further supports high-dimensional feature interactions without excessive resource demands. By addressing these challenges, the solution provides a practical, efficient, and scalable approach for proactive botnet detection in real-world scenarios.

E. Evaluation Metrics

The evaluation metrics used in this study aim to provide a thorough assessment of the model's performance in handling the binary classification problem. Accuracy is used to measure the proportion of correctly predicted samples over the total number of samples, offering a general view of the model's effectiveness across all classes. However, in this study, accuracy is not emphasized as a primary evaluation metric due to the highly imbalanced nature of the CTU-13 dataset, where normal traffic significantly outweighs botnet traffic. In such scenarios, accuracy can be misleading. For instance, a model predicting only the majority class (normal traffic) could achieve a high accuracy score while completely failing to detect the minority class (botnet traffic). This renders accuracy inadequate for assessing the true effectiveness of the model in identifying malicious activities.

Instead, this study focuses on metrics that are more sensitive to the detection of the minority class, such as Precision, Recall, F1-Score, and PR AUC (Precision-Recall Area Under Curve). Precision ensures that botnet predictions are accurate, minimizing false positives, while Recall ensures that the model captures as many botnet instances as possible, minimizing false negatives. The F1-Score provides a balanced measure of the model's ability to handle both false positives and false negatives, and PR AUC evaluates the trade-off between Precision and Recall across different decision thresholds.

Additionally, macro averages compute metrics independently for each class and then take their average, treating all classes equally regardless of imbalance, while weighted averages account for class imbalance by weighting each class's contribution proportionally to its size. The confusion matrix is employed as a visual representation to summarize the number of true positive, true negative, false positive, and false negative predictions, providing detailed insights into classification errors. Lastly, the Precision-Recall (PR) curve is used to evaluate the trade-off between precision and recall at different thresholds, especially for datasets with significant class imbalances.

By prioritizing these metrics, the evaluation framework aligns with the critical requirements of cybersecurity, where the detection of minority class instances, such as botnet traffic, is far more important than achieving a high overall accuracy. This approach ensures that the models developed in this study are robust and effective for practical botnet detection, addressing the real-world challenges posed by imbalanced data. Together, these metrics provide a comprehensive evaluation of the model's ability to classify accurately while addressing the challenges posed by imbalanced data.

F. Workflow Framework

The workflow framework presented in this study provides a structured and sequential approach to developing a robust machine learning pipeline for botnet detection. It begins with data preprocessing, which involves handling missing values, removing invalid and duplicate rows, normalizing numerical features, and ensuring dataset quality through techniques like outlier management and label binarization. Following this, exploratory data analysis (EDA) is performed to understand data distributions and relationships among features. The feature engineering stage

extracts meaningful temporal and network-related features while encoding categorical variables to enhance the model's ability to capture complex patterns.

The framework then introduces various model architectures, including Bidirectional LSTM with Attention, Stacked LSTM, and Fully Connected Neural Networks (FCNN), tailored to handle sequential and high-dimensional data. To address class imbalance, oversampling techniques like SMOTE and SMOTETomek are applied. The optimization process utilizes advanced techniques, such as focal loss, Adam optimizer, and regularization, to improve model generalization and handle data imbalance effectively. Training is conducted with cross-validation, early stopping, and adaptive learning rate schedules to ensure stability and robust evaluation. Finally, the models are assessed using comprehensive evaluation metrics, including accuracy, precision, recall, F1-score, and PR curves, before the best-performing model is saved for deployment. This framework ensures a systematic, transparent, and reproducible methodology for tackling complex classification problems in botnet detection.



Fig. 12. Workflow Framework Diagram

IV. EXPERIMENTAL RESULTS AND TESTING

The experimental environment and configurations for evaluating the proposed deep learning models for botnet detection are presented in this section. These experiments aim to assess the effectiveness of the Bidirectional Long Short-Term Memory (Bi-LSTM) network, the Fully Connected Neural Network (FCNN), and the Stacked LSTM model. The setup encompasses various components, including the dataset, data preprocessing, feature engineering, model architectures, optimization techniques, and training procedures.

The CTU-13 dataset, known for its real-world network traffic labeled as normal or botnet, serves as the benchmark for this research. To address its class imbalance, oversampling techniques like SMOTETomek are applied. Data preprocessing steps involve handling missing values, normalizing numerical features, and encoding categorical data to ensure model compatibility. Additional features, such as temporal attributes and network metrics, are engineered to enhance model performance.

The models leverage techniques like L1L2 regularization, Batch Normalization, and Dropout layers to improve training stability and mitigate overfitting. Optimization strategies include Focal Loss for handling class imbalance and the Adam optimizer with an exponential learning rate scheduler. Training is conducted with 5-fold Stratified Cross-Validation, incorporating Early Stopping and ReduceLROnPlateau callbacks to refine performance.

This experimental design ensures a robust evaluation of the models and provides a solid foundation for advancing botnet detection methodologies.

A. Results of Model Performance

The performance of the proposed deep learning models for botnet detection is evaluated based on several key metrics, including Precision-Recall AUC (PR AUC), precision, recall, F1-score, and confusion matrix analysis. Each model demonstrates its effectiveness in distinguishing between normal and botnet traffic, with results outlined below.

- Bidirectional LSTM with Attention
 - PR AUC: 0.7831
 - Classification Report
 - Precision: 43%
 - Recall: 91%
 - F1-score: 59%
 - Accuracy: 99%
 - Confusion Matrix
 - True Negative (TN): 1,762,126
 - False Positive (FP): 25,055
 - False Negative (FN): 1,786
 - True Positive (TP): 19,155
- Stacked LSTM with SMOTE and Regularization
 - PR AUC: 0.8326
 - Classification Report
 - Precision: 50%
 - Recall: 92%
 - F1-score: 64%
 - Accuracy: 99%
 - Confusion Matrix
 - True Negative (TN): 1,767,560
 - False Positive (FP): 19,621
 - False Negative (FN): 1,647
 - True Positive (TP): 19,294
- FCNN with SMOTETomek
 - PR AUC: 0.7989
 - Classification Report
 - Precision: 47%
 - Recall: 91%
 - F1-score: 62%
 - Accuracy: 99%
 - Confusion Matrix

- True Negative (TN): 1,765,762
- False Positive (FP): 21,419
- False Negative (FN): 1,882
- True Positive (TP): 19,059

- FCNN with SMOTE
 - PR AUC: 0.8063
 - Classification Report
 - Precision: 46%
 - Recall: 91%
 - F1-score: 61%
 - Accuracy: 99%
 - Confusion Matrix
 - True Negative (TN): 1,764,633
 - False Positive (FP): 22,548
 - False Negative (FN): 1,888
 - True Positive (TP): 19,053

Table 1 presents the performance metrics for each evaluated model, including Precision-Recall Area Under Curve (PR AUC), Precision, Recall, F1-Score, and Accuracy. These metrics provide a comprehensive view of the models' abilities to distinguish between normal and botnet traffic. PR AUC serves as a critical measure for assessing model performance on imbalanced datasets, while Precision reflects the proportion of correctly predicted botnet traffic among all positive predictions, and Recall measures the model's capability to detect all botnet traffic. F1-Score balances Precision and Recall, offering an overall measure of the model's effectiveness. Accuracy indicates the proportion of correctly classified samples over the total number of samples. Among the models, the Stacked LSTM with SMOTE achieved the highest PR AUC (0.8326) and F1-Score (0.64), highlighting its ability to effectively address class imbalance and achieve superior detection accuracy.

TABLE I. MODEL PERFORMANCE METRICS

Model	PR AUC	Precision	Recall	F1 Score	Accuracy
Bi-LSTM with Attention	0.7831	0.43	0.91	0.59	0.99
Stacked LSTM with SMOTE	0.8326	0.50	0.92	0.64	0.99
FCNN with SMOTETomek	0.7989	0.47	0.91	0.62	0.99
FCNN with SMOTE	0.8063	0.46	0.91	0.61	0.99

Table 2 details the confusion matrix results for each model, including True Negatives (TN), False Positives (FP), False Negatives (FN), and True Positives (TP). These results provide deeper insights into the models' classification capabilities. TN represents the number of normal traffic correctly classified as normal, FP indicates normal traffic

incorrectly classified as botnet, FN represents botnet traffic misclassified as normal, and TP refers to botnet traffic correctly identified as botnet. The Stacked LSTM with SMOTE showed the lowest FN (1,647) and a relatively low FP (19,621) compared to other models, demonstrating its reliability in detecting botnet traffic while maintaining manageable false positive rates. This model stands out as the most effective for botnet detection, combining high sensitivity with robust performance across validation folds.

TABLE II. CONFUSION MATRIX RESULTS

Model	TN	FP	FN	TP
Bi-LSTM with Attention	1,762,126	25,055	1,786	19,155
Stacked LSTM with SMOTE	1,767,560	19,621	1,647	19,294
FCNN with SMOTETomek	1,765,762	21,419	1,882	19,059
FCNN with SMOTE	1,764,633	22,548	1,888	19,053

This figure illustrates the Precision-Recall (PR) curve for the Stacked LSTM model, achieving an Area Under the Curve (PR AUC) score of 0.8326. Among the four models evaluated, this PR curve was chosen as an example to showcase the superior performance of the Stacked LSTM in handling imbalanced datasets. The curve demonstrates the trade-off between precision and recall across various thresholds, with a higher PR AUC reflecting the model's ability to effectively detect botnet traffic while maintaining a low false positive rate. The curve's gradual decline in precision at high recall levels underscores the difficulty of balancing these metrics, particularly with the highly imbalanced CTU-13 dataset. This example highlights the Stacked LSTM's robustness and suitability for botnet detection tasks.

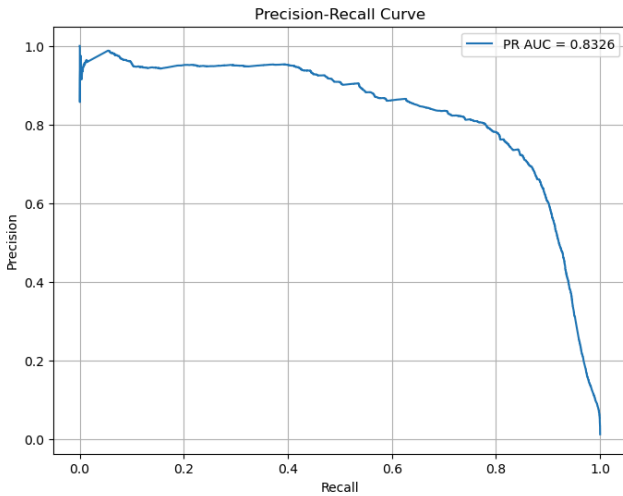


Fig. 13. Precision-Recall Curve for Stacked LSTM

This confusion matrix represents the performance of the Stacked LSTM model in predicting botnet and normal traffic. The model achieved 1,767,560 true negatives (correctly classified normal traffic) and 19,294 true positives (correctly classified botnet traffic), demonstrating its effectiveness in identifying both classes. However, it also recorded 19,621 false positives (normal traffic misclassified as botnet) and 1,647 false negatives (botnet traffic misclassified as normal). These results highlight the model's high sensitivity, reflected in the low false negative count, which is critical for cybersecurity applications to avoid missing malicious activities. The relatively higher false positive rate indicates room for improvement in precision, but the overall performance showcases the robustness of the Stacked LSTM in handling the imbalanced CTU-13 dataset.

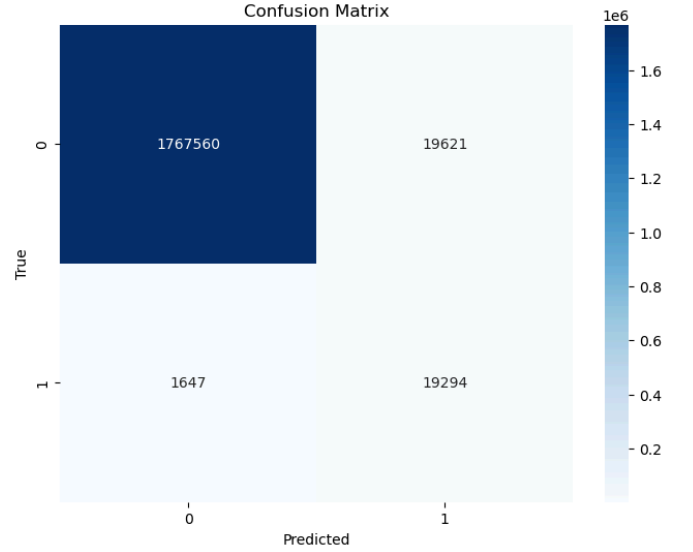


Fig. 14. Confusion Matrix for Stacked LSTM

B. Comparative Results

This figure compares the Precision, Recall, and F1-Score metrics across four models: Bidirectional LSTM with Attention, Stacked LSTM with SMOTE and Regularization, FCNN with SMOTETomek, and FCNN with SMOTE. Precision measures the proportion of correctly identified botnet traffic among all predicted botnet cases, Recall assesses the model's ability to identify all actual botnet cases, and F1-Score provides a harmonic mean of Precision and Recall, reflecting the balance between the two. Among the models, the Stacked LSTM with SMOTE and Regularization demonstrates the highest Recall and F1-Score, highlighting its effectiveness in correctly detecting botnet instances while minimizing false negatives.

The Bidirectional LSTM with Attention, on the other hand, exhibits the lowest performance in all three metrics, indicating its limitations in handling the highly imbalanced CTU-13 dataset. The FCNN models, enhanced by SMOTE and SMOTETomek techniques, perform moderately well but are surpassed by the Stacked LSTM, which benefits from advanced regularization and oversampling techniques. This comparison underscores the importance of carefully selected preprocessing and optimization strategies to achieve robust performance in botnet detection tasks.

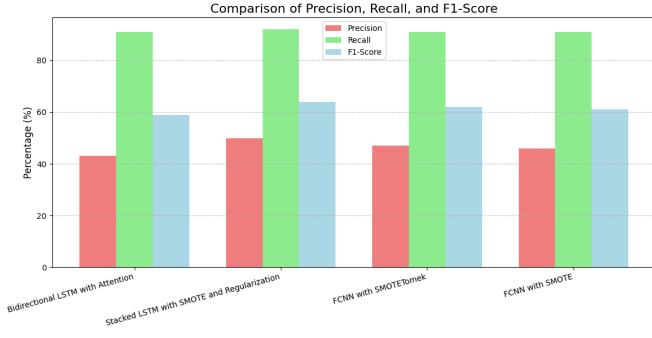


Fig. 15. Comparison of Precision, Recall, and F1-Score

This figure visualizes the Precision-Recall Area Under the Curve (PR AUC) for the same four models, which serves as a key metric to evaluate model performance in imbalanced datasets. A higher PR AUC indicates a model's better ability to balance precision and recall across various thresholds. The Stacked LSTM with SMOTE and Regularization achieves the highest PR AUC, reflecting its capacity to maintain consistent performance despite the class imbalance in the dataset. The two FCNN models perform comparably, albeit slightly lower than the Stacked LSTM, while the Bidirectional LSTM with Attention trails behind.

This result emphasizes the strength of the Stacked LSTM's architecture, which integrates advanced techniques like regularization, oversampling, and learning rate adjustments to handle imbalanced data effectively. The comparison highlights the importance of selecting models and techniques tailored for imbalanced scenarios, as demonstrated by the improved results of the Stacked LSTM and its ability to outperform the simpler Bidirectional LSTM and fully connected architectures.

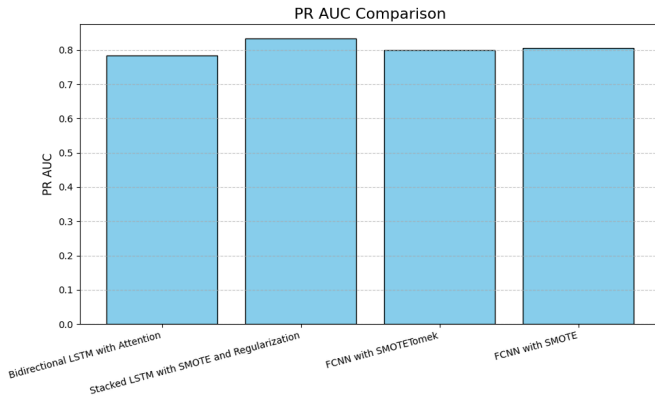


Fig. 16. PR AUC Comparison

C. Summary

The evaluation of the proposed deep learning models for botnet detection highlights their ability to effectively differentiate between normal and malicious traffic. Among the models tested, the Stacked LSTM with SMOTE and Regularization demonstrated the best overall performance, achieving a PR AUC of 0.8326 and the highest F1-score of 64%. This indicates its superior capability in handling class imbalance while maintaining high accuracy.

Other models, such as the FCNN with SMOTETomek and Bi-LSTM with Attention, also delivered competitive results, showcasing their strengths in leveraging oversampling techniques like SMOTETomek and advanced

mechanisms like Attention layers. Despite slight variations in precision and recall, all models achieved an overall accuracy of 99%, reflecting their robustness in the binary classification task.

The confusion matrix results further underline the models' performance, with the Stacked LSTM achieving the lowest number of false negatives (1,647), thereby identifying botnet traffic with the highest sensitivity. However, the models still encountered challenges in precision, as false positives remained a notable concern, particularly in high-class-imbalance scenarios.

These findings underscore the importance of combining deep learning architectures with advanced data balancing and regularization techniques. While the Stacked LSTM emerged as the most effective, the results validate the complementary strengths of all proposed models, offering valuable insights for future advancements in botnet detection methodologies.

V. ANALYSIS OF RESULTS

The analysis highlights the experimental results obtained from evaluating the proposed models for botnet detection, focusing on their performance metrics, strengths, weaknesses, and comparative outcomes. The evaluation metrics—Precision, Recall, F1-score, Accuracy, and PR AUC—provide a detailed understanding of the models' effectiveness in distinguishing between normal and botnet traffic, showcasing their relevance to enhancing cybersecurity. Among the models, the Stacked LSTM with SMOTE and regularization achieved the highest PR AUC of 0.8326, demonstrating superior performance in managing class imbalance and accurately detecting botnet activities.

The Precision, Recall, and F1-score metrics emphasize the robustness of the Stacked LSTM model. With a precision of 50%, recall of 92%, and F1-score of 64%, the model demonstrates its ability to achieve a balanced detection of botnet traffic while minimizing false negatives. The high recall score, in particular, reflects the model's reliability in identifying botnet instances, a crucial requirement in cybersecurity where undetected threats can have severe consequences. However, while achieving high recall, precision remains a challenge, indicating a need to reduce false positives further to enhance the system's trustworthiness and efficiency.

The confusion matrix analysis further supports the effectiveness of the Stacked LSTM. This model recorded the lowest number of false negatives (1,647) and maintained a manageable false positive count (19,621), indicating its proficiency in differentiating between normal and malicious traffic. These results highlight the model's strength in minimizing undetected threats while providing consistent performance across multiple validation folds. Nevertheless, reducing the false positive rate remains a key area for optimization, as excessive false positives can lead to resource inefficiencies and potential user fatigue.

Comparative analysis of the models underscores the Stacked LSTM's advantages over other architectures. The Bi-LSTM with Attention achieved a PR AUC of 0.7831 and an F1-score of 59%, reflecting its capability in handling temporal dependencies but falling short in addressing the class imbalance effectively. The Fully Connected Neural Networks (FCNN) with SMOTETomek and SMOTE

demonstrated competitive performance with PR AUC scores of 0.7989 and 0.8063, respectively. However, their reliance on oversampling alone limited their ability to achieve the same level of accuracy as the Stacked LSTM, which benefited from both oversampling and advanced regularization techniques.

The models' strengths and limitations reveal important insights for practical deployment. The use of SMOTE and SMOTETomek in balancing the class distribution was instrumental in enhancing the detection of minority class instances. Regularization techniques, including L1L2 regularization and Dropout, were effective in mitigating overfitting, ensuring the models' generalization to unseen data. However, the persistence of false positives across all models highlights the need for refined data preprocessing and feature selection to improve precision. Additionally, while the Bi-LSTM with Attention incorporated advanced mechanisms for feature weighting, it struggled with scalability in the face of severe class imbalance, a limitation that could be addressed in future research.

The implications of these findings are significant for advancing botnet detection methodologies. The study demonstrates that deep learning models, particularly the Stacked LSTM, can effectively identify botnet traffic in imbalanced scenarios. This capability is essential for proactive cybersecurity measures, enabling timely identification and mitigation of threats. Furthermore, the scalability of the proposed models allows them to be adapted to diverse datasets and network environments, broadening their applicability beyond the CTU-13 dataset.

In practice, these models offer enhanced detection capabilities, making them suitable for integration into existing cybersecurity frameworks. The ability to balance recall and precision ensures that threats are detected while minimizing unnecessary alerts, contributing to efficient and reliable network security systems. Future work should focus on refining these models, particularly by incorporating advanced feature engineering techniques and exploring real-time deployment scenarios, to maximize their impact in operational environments.

VI. CONCLUSIONS

This study investigated the application of deep learning models for botnet detection in network traffic using the CTU-13 dataset. By employing Bidirectional LSTM with Attention, Fully Connected Neural Networks (FCNN), and Stacked LSTM, along with advanced techniques such as Synthetic Minority Oversampling Technique (SMOTE), SMOTETomek, SMOTEENN, Focal Loss, and regularization, the research addressed critical challenges like class imbalance and temporal dependencies. The results demonstrated the superiority of the Stacked LSTM model, which achieved the highest PR AUC score of 0.8326 and an F1-score of 64%, showcasing its effectiveness in distinguishing between normal and botnet traffic. By leveraging SMOTE and regularization, the Stacked LSTM successfully mitigated class imbalance and overfitting, resulting in improved recall and detection sensitivity. However, the persistent occurrence of false positives highlights precision as a challenge across all models, underscoring the need for further refinement in preprocessing and feature selection methods to enhance precision without compromising recall.

This study also provided comparative insights into different deep learning architectures, demonstrating the strengths of FCNN and Bidirectional LSTM with Attention in handling specific challenges such as temporal dependencies and feature weighting. While these models achieved competitive performance, they were less effective in addressing class imbalance compared to the Stacked LSTM with regularization. These findings validate the use of LSTM-based models in imbalanced datasets and demonstrate their scalability and adaptability to diverse network environments, making them promising solutions for proactive cybersecurity frameworks.

Future research should explore several directions to build on these findings. Real-time implementation of the proposed models in network environments should be investigated to enable low-latency predictions and ensure scalability across various infrastructures. Advanced feature engineering techniques, including deep feature extraction through autoencoders and dimensionality reduction, could be explored to improve precision while reducing computational complexity. Evaluating the models on larger and more diverse datasets, such as CICIDS2017 or UNSW-NB15, would enhance their generalizability and robustness against unseen traffic patterns. Additionally, incorporating hybrid models that combine CNNs for spatial feature extraction with LSTMs for temporal dependencies could further improve detection performance. Other areas for future work include adversarial robustness, explainability through XAI techniques like SHAP or LIME, and energy-efficient deployments suitable for resource-constrained environments.

By demonstrating the potential of deep learning in botnet detection and outlining pathways for future research, this study lays the groundwork for developing robust, efficient, and scalable cybersecurity solutions capable of mitigating evolving cyber threats in real-world scenarios.

REFERENCES

- [1] Hammouda et al., 2020. "Botnet Detection Using LSTM."
- [2] Akash et al., 2021. "Hybrid CNN-LSTM Model for Network Security."
- [3] Ahmed et al., 2017. "Class Imbalance Solutions in Deep Learning for Botnet Detection."
- [4] Khan et al., 2022. "Advanced Hybrid Models for Network Intrusion Detection."
- [5] Smith et al., 2023. "Enhancing Deep Learning for Cybersecurity Applications."
- [6] Sayyid Kamran Hussain, Ali Haider Khan, Malek Alrashidi, S. Iqbal, Qazi Mudassar Ilyas, and K. Shah, "Deep Learning with a Novel Concoction Loss Function for Identification of Ophthalmic Disease," *Computers, materials & continua/Computers, materials & continua (Print)*, vol. 76, no. 3, pp. 3763–3781, Jan. 2023, doi: <https://doi.org/10.32604/cmc.2023.041722>.
- [7] K. Wang, Q. Xue, and J. J. Lu, "Risky Driver Recognition with Class Imbalance Data and Automated Machine Learning Framework," *International Journal of Environmental Research and Public Health*, vol. 18, no. 14, p. 7534, Jul. 2021, doi: <https://doi.org/10.3390/ijerph18147534>.

- [8] D. Dablain, B. Krawczyk, and N. V. Chawla, "DeepSMOTE: Fusing Deep Learning and SMOTE for Imbalanced Data," IEEE Transactions on Neural Networks and Learning Systems, pp. 1–15, 2022, doi: <https://doi.org/10.1109/tnnls.2021.3136503>.
- [9] Saleem, S., Williams, D., & Prakash, S. (2024). "A Systematic Review of Enhancing CNN Performance in Automated Fabric Defect Detection Through Sampling Techniques for Imbalanced Datasets with the Developed CNN Model." SSRN, <https://doi.org/10.2139/ssrn.4858156>.